

深度学习入门

WPC

2025 年 8 月 8 日

目录

1	感知机	3
1.1	感知机简介	3
1.2	实现与门	3
1.3	局限性	4
2	神经网络	6
2.1	神经网络简介	6
2.2	损失函数	7
3	计算图	8
3.1	计算图简介	8
3.2	反向传播	8
4	CNN	10
4.1	CNN 简介	10
4.2	卷积层	10
4.3	池化层	12
5	自然语言处理	13
5.1	单词表示	13
5.2	基于同义词词典的方法	13
5.3	基于统计的方法	13
5.4	基于推理的方法	14
6	分词	15
6.1	分词简介	15
6.2	BPE	15
6.3	BBPE	16

目录	2
6.4 WordPiece	16
7 RNN	17
7.1 语言模型	17
7.2 RNN 介绍	17
7.3 语言模型的评估	18
7.4 梯度消失和梯度爆炸	19
7.5 LSTM	19
8 文本生成	21
8.1 Seq2Seq 模型	21
8.2 Self-Attention	21
9 Transformer	24
9.1 位置编码	24
9.2 编码器与解码器	24

1 感知机

1.1 感知机简介

感知机 (Perceptron) 由美国学者 Frank Rosenblatt 于 1957 年提出, 作为神经网络最早期的算法之一, 感知机奠定了深度学习发展的基础, 至今仍具有重要的学习和研究价值。

图 1.1 是一个接收两个输入信号的感知机的例子。感知机接收多个输入信号, 并输出一个信号, 在感知机模型中, 输入与输出的信号通常是二值信号 (0/1)。 x_1 、 x_2 是输入信号, y 是输出信号, w_1 、 w_2 是对应权重。图中的 $\bigcirc$ 代表“神经元”。输入信号被送往神经元时, 会被分别乘以固定的权重, 神经元会计算传送过来的信号的总和, 当这个总和超过了某个阈值 θ , 神经元输出 1, 这也被称为“神经元被激活”。

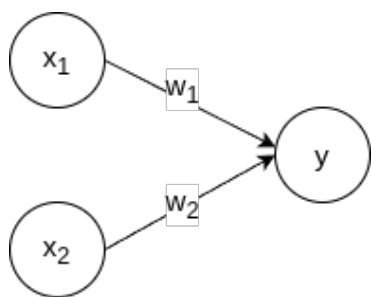


图 1.1: 感知机示例

把上述内容用数学式来表示。

$$y = \begin{cases} 0, & w_1x_1 + w_2x_2 \leq \theta \\ 1, & w_1x_1 + w_2x_2 > \theta \end{cases}$$

1.2 实现与门

下面考虑用感知机来实现与门, 表 1.1 是与门的真值表。

x_1	x_2	$y = x_1 \wedge x_2$
0	0	0
0	1	0
1	0	0
1	1	1

表 1.1: 与门的真值表

我们需要做的就是确定能满足真值表的 w_1 、 w_2 、 θ 的值。实际上, 参数的选择方法有无数多种。比如, 当 $(w_1, w_2, \theta) = (0.5, 0.5, 0.7)$ 时, 可以满足条件。此外, 当 $(w_1, w_2, \theta) = (0.5, 0.5, 0.8)$

或者 (1.0, 1.0, 1.0) 时, 同样也满足与门的条件。设定这样的参数后, 仅当 x_1 和 x_2 同时为 1 时, 信号的加权总和才会超过给定的阈值 θ 。

为了数学表达的简洁性, 我们先把上面感知机实现的数学式子改写一下, 用偏置 b 代替阈值 θ , 注意 $b = -\theta$ 。

$$y = \begin{cases} 0, & b + w_1x_1 + w_2x_2 \leq 0 \\ 1, & b + w_1x_1 + w_2x_2 > 0 \end{cases}$$

1.3 局限性

前面我们用感知机实现了与门, 现在, 我们来探讨一个更具挑战性的例子——异或门。表 1.2 是异或门的真值表, 应该设定什么样的参数能满足这个真值表呢?

x_1	x_2	$y = x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

表 1.2: 异或门的真值表

实际上, 用前面介绍的感知机是无法实现这个异或门的。思考感知机如何实现与门的, 其实是感知机生成一条直线来划分空间, 以我们之前实现的与门为例, 生成的直线为 $-0.8 + 0.5x_1 + 0.5x_2 = 0$ 。

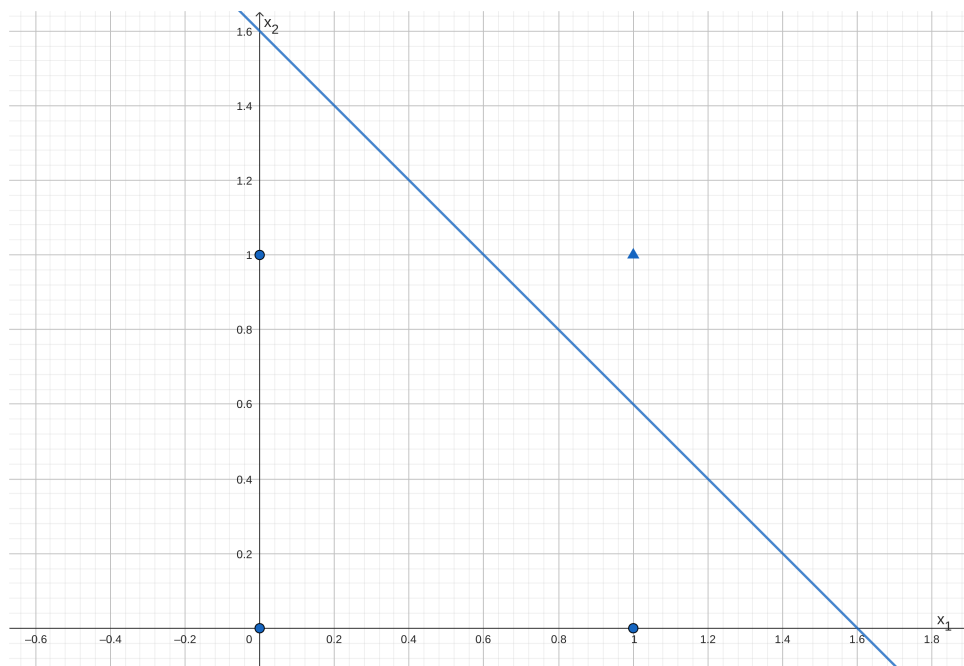


图 1.2: 与门可视化

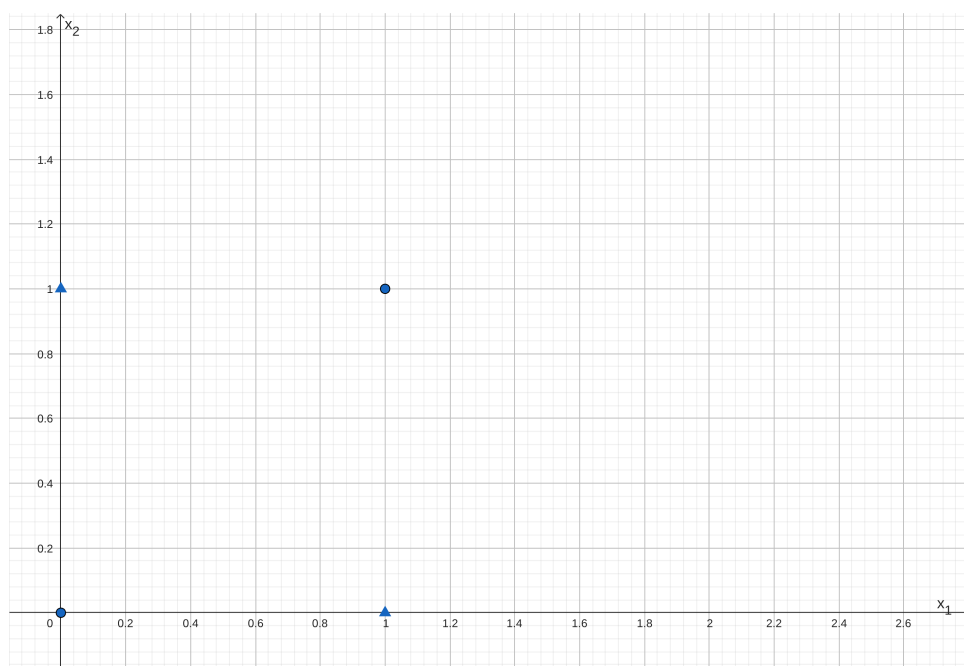


图 1.3: 异或门可视化

想要用一条直线将图 1.3 中的数据分开，是做不到的。感知机只能表示线性可分的空间，不过，就如同我们能通过与门、与非门、或门构建异或门，我们也能够通过“多层叠加”感知机来处理非线性可分的问题。

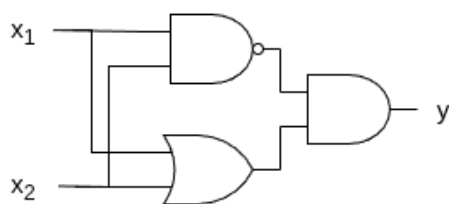


图 1.4: 通过组合实现异或门

2 神经网络

2.1 神经网络简介

在前面的感知机模型中，参数的确定并不是由计算机自动完成的，而是由我们人为地根据真值表等“训练数据”手动计算得出。为了让计算机能够自动学习和调整参数，神经网络的概念应运而生。

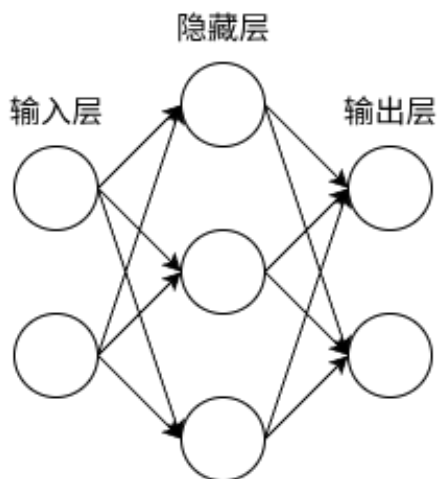


图 2.1: 神经网络示例

将之前感知机的函数改写成以下更简洁的形式。

$$y = h(b + w_1x_1 + w_2x_2) \quad h(x) = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$$

$h(x)$ 函数会将输入信号的总和转换为输出信号，被称为激活函数 (activation function)。我们之前使用的激活函数是“阶跃函数”，一旦输入超过阈值，就切换输出。除了这个外，神经网络中常用的激活函数有 sigmoid 函数、ReLU (Rectified Linear Unit) 函数。

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \text{ReLU}(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

对于输出层，我们有时不是直接把输出层的值当作最终结果，比如如果需要做个多分类问题，可能需要用 softmax 函数对输出层处理。

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

2.2 损失函数

神经网络“从数据中学习”，由数据自动决定权重参数的值。为了达成这个目的，我们先要决定损失函数（loss function），之后，神经网络就以这个指标为基准，寻找最优权重参数。常用的损失函数有均方误差（mean squared error）和交叉熵误差（cross entropy error）。

$$\text{MSE} = \frac{1}{2} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \text{CE} = - \sum_{i=1}^K \hat{y}_i \log(y_i)$$

这里， y_i 是表示神经网络的输出， $\hat{y}_i$ 表示监督数据， i 表示数据的维数。交叉熵误差主要用于多分类问题，实际上他只计算对应正确解标签的输出，假设正确解标签的索引是“1”，与之对应的神经网络的输出是 0.7，则交叉熵误差是 $-\log 0.7$ 。

使用数据进行学习，就是针对训练数据计算损失函数的值，找出使该值尽可能小的参数。如果遇到大数据，数据量会有几百万、几千万之多，这种情况下以全部数据为对象计算损失函数是不现实的。因此，我们从全部数据中选出一部分，作为全部数据的“近似”。从训练数据中选出一批数据（mini-batch），然后对每个 mini-batch 进行学习，这种学习方式称为 mini-batch 学习。

学习的目标是最小化损失函数，梯度法是实现这一目标的常用优化方法：

$$\begin{aligned} x_1^{(t+1)} &= x_1^{(t)} - \eta \frac{\partial f}{\partial x_1}(x_1^{(t)}, x_2^{(t)}) \\ x_2^{(t+1)} &= x_2^{(t)} - \eta \frac{\partial f}{\partial x_2}(x_1^{(t)}, x_2^{(t)}) \end{aligned}$$

η 被称为学习率（learning rate）。可以通过数值微分（Numerical Differentiation）来计算导数，即向 h 中赋入一个微小值，直接计算导数式。

$$\frac{d}{dx} f(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

3 计算图

3.1 计算图简介

计算图 (Computational Graph) 是一种将复杂的数学表达式分解为基本运算单元并用图结构表示的方法。在计算图中, 节点 (Node) 表示变量或操作 (如加法、乘法、激活函数等), 边 (Edge) 表示数据在操作之间的流动或依赖关系。

假设小明在超市买了两个 5 元/个的苹果, 消费税为 10%, 请计算他需要支付的总金额。

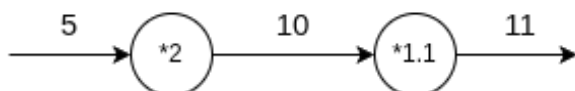


图 3.1: 计算图示例

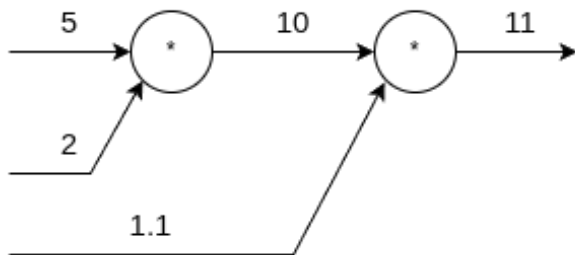


图 3.2: 计算图示例

用计算图解题的情况下, 需要先构建计算图, 然后在计算图上, 从左向右进行计算。这里的“从左向右进行计算”被称为正向传播 (forward propagation)。事实上, 也存在反向传播 (backward propagation), 反向传播将在接下来的导数计算中发挥重要作用。

计算图的特征是可以通过传递“局部计算”获得最终结果, 这就带来一个优点: 无论全局是多么复杂的计算, 都可以通过局部计算使各个节点致力于简单的计算, 从而简化问题。实际上, 使用计算图最大的原因是, 可以通过反向传播高效计算导数。

3.2 反向传播

我们再来思考一下之前的问题, 我们计算了购买 2 个苹果时加上消费税最终需要支付的金额。这里, 假设我们想知道苹果价格的上涨会在多大程度上影响最终的支付金额, 即求“支付金额关于苹果的价格的导数”。设苹果的价格为 x , 支付金额为 L , 则相当于求 $\frac{dL}{dx}$ 。这个导数的值表示当苹果的价格稍微上涨时, 支付金额会增加多少。

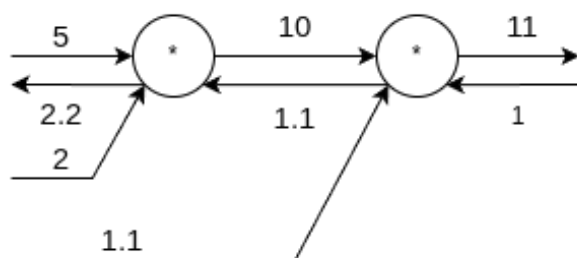


图 3.3: 反向传播示例

反向传播传递“局部导数”，将导数的值写在箭头的下方。在这个例子中，反向传播从右向左传递导数的值（ $1 \rightarrow 1.1 \rightarrow 2.2$ ）。从这个结果中可知，“支付金额关于苹果的价格的导数”的值是 2.2。这意味着，如果苹果的价格上涨 1 元，最终的支付金额会增加 2.2 元。可以看出，计算图的优点是，可以通过反向传播高效地计算各个变量的导数值。

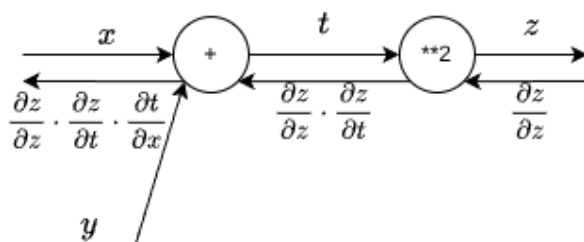


图 3.4: 反向传播示例

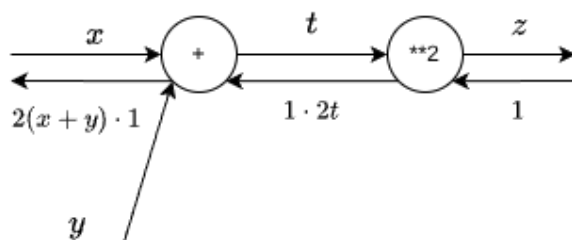


图 3.5: 反向传播示例

计算图的反向传播是基于链式法则成立的，通过提前计算每个节点相对输入的偏导数，可以快速进行反向传播。

4 CNN

4.1 CNN 简介

本章的主题是卷积神经网络 (Convolutional Neural Network, CNN)，之前介绍的神经网络中，相邻层的所有神经元之间都有连接，这称为全连接网络 (fully-connected Network)。

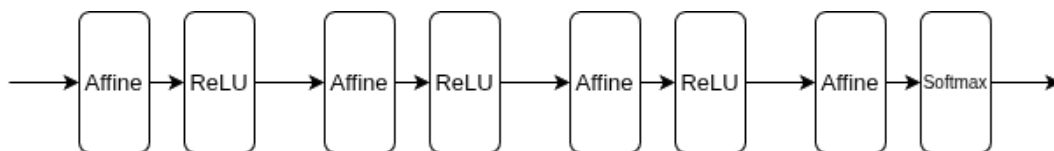


图 4.1: 全连接网络示例

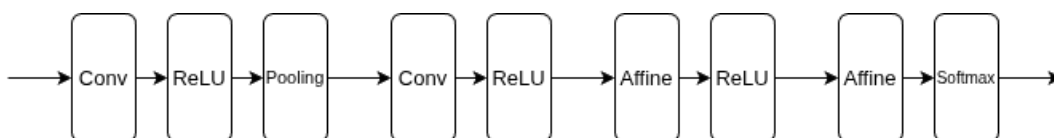


图 4.2: 卷积神经网络示例

相较全连接网络，CNN 中新出现了卷积层 (Convolution 层) 和池化层 (Pooling 层)，CNN 的层的连接顺序是 “Conv - ReLU - Pooling” (Pooling 层有时会被省略)。还需要注意的是，在 CNN 中，靠近输出的层中使用了之前的 “Affine - ReLU” 组合。此外，最后的输出层中使用了之前的 “Affine - Softmax” 组合，这些都是 CNN 中比较常见的结构。

4.2 卷积层

全连接层 (Affine 层) 存在什么问题呢？那就是数据的形状被 “忽视” 了。比如，输入数据是图像时，图像通常是高、长、rgb 通道方向上的 3 维形状。但是，向全连接层输入时，需要将 3 维数据拉平为 1 维数据。图像是 3 维形状，这个形状中含有重要的空间信息。比如，空间上邻近的像素为相似的值，形状中可能隐藏有值得提取的特征。

卷积层可以保持形状不变。当输入数据是图像时，卷积层会以 3 维数据的形式接收输入数据，并同样以 3 维数据的形式输出至下一层。利用卷积层，有可能正确理解输入数据的形状的信息，卷积层的输入输出数据称为特征图 (feature map)。其中，输入数据称为输入特征图 (input feature map)，输出数据称为输出特征图 (output feature map)。

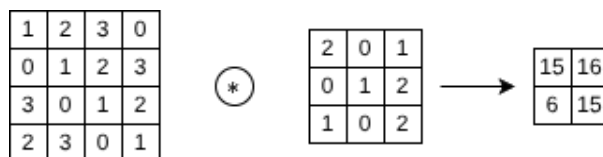


图 4.3: 卷积运算示例

卷积层进行的处理就是卷积运算，相当于图像处理中的“滤波器运算”，滤波器也被称为卷积核，卷积运算以一定间隔滑动滤波器的窗口并应用，这个计算也被称为乘积累加运算。

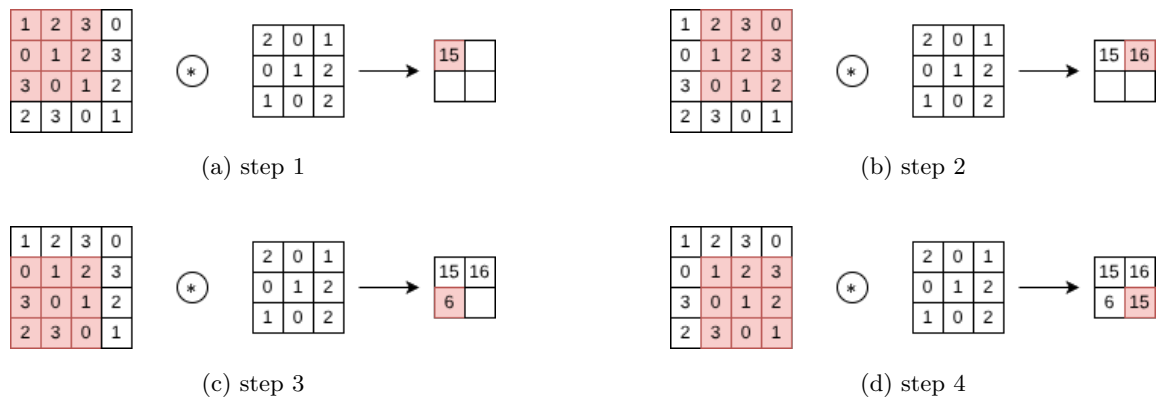


图 4.4: 卷积运算流程说明

CNN 中也存在偏置，偏置通常只有 1 个，这个值会被加到应用了滤波器的所有元素上。

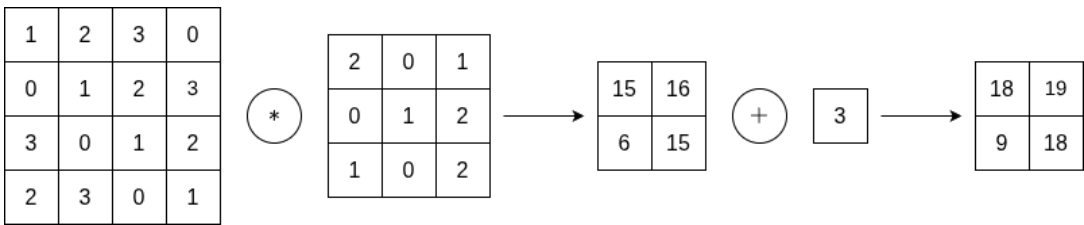


图 4.5: 偏置示例

在进行卷积层的处理之前，有时要向输入数据的周围填入固定的数据（比如 0），这称为填充 (padding)，主要是为了调整输出的大小。如下图所示，通过应用了幅度为 1 的填充，大小为 (4, 4) 的输入数据变成了 (6, 6) 的形状，然后，应用大小为 (3, 3) 的滤波器，生成了大小为 (4, 4) 的输出数据。

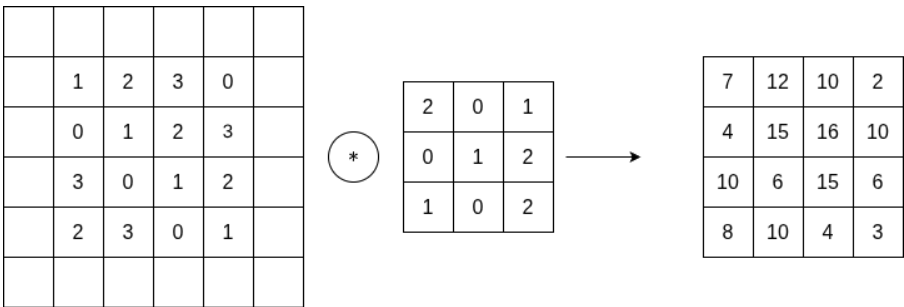


图 4.6: 填充示例

应用滤波器的位置间隔称为步幅 (stride)。下图为一个步幅为 2 卷积运算示例。

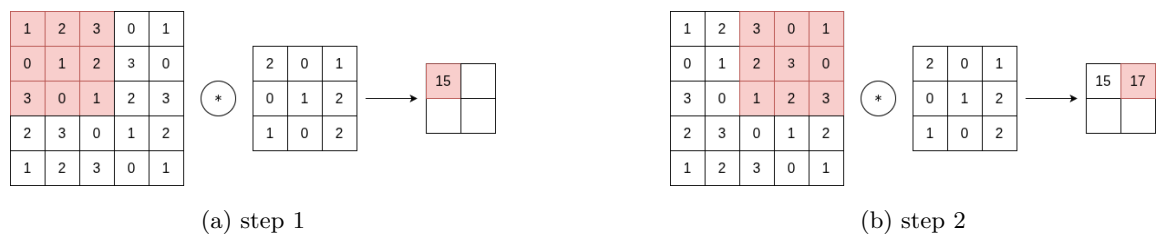


图 4.7: 步幅示例

图像是 3 维数据，除了高、长方向之外，还需要处理通道方向，在 3 维数据的卷积运算中，输入数据和滤波器的通道数要设为相同的值。最后数据输出是 1 张特征图，如果要数据输出在通道方向上也拥有维度，那么就需要用到多个滤波器，生成多个特征图。

4.3 池化层

池化是缩小高、长方向上的空间的运算，下图是按步幅 2 进行 2×2 的 Max 池化时的处理顺序。一般来说，池化的窗口大小会和步幅设定成相同的值。除了 Max 池化之外，还有 Average 池化、随机池化等，在图像识别领域，主要使用 Max 池化。

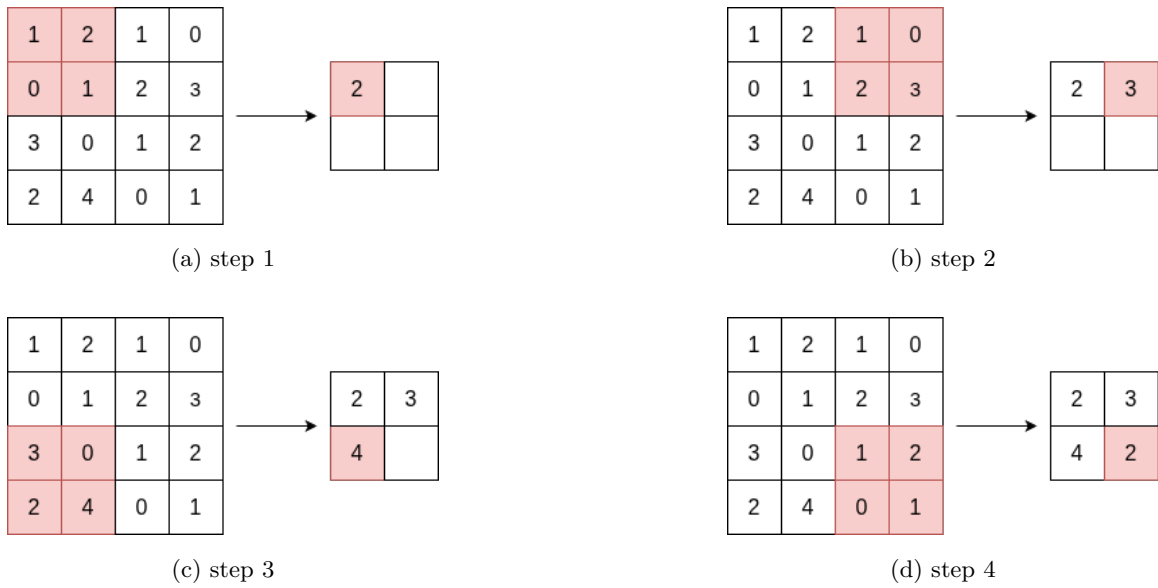


图 4.8: 池化层流程说明

池化层的特征

- 没有要学习的参数: 池化层和卷积层不同，没有要学习的参数。
- 通道数不发生变化: 经过池化运算，输入数据和输出数据的通道数不会发生变化。
- 对微小的位置变化具有鲁棒性: 输入数据发生微小偏差时，池化仍会返回相同的结果。

5 自然语言处理

5.1 单词表示

自然语言处理 (Natural Language Processing, NLP)，它的目标就是让计算机理解人说的话，进而帮助人们更好地完成某些事情。语言是由文字构成的，而语言的含义是由单词构成的，即单词是含义的最小单位。所以，为了让计算机理解自然语言，第一步就是让它理解单词含义。

5.2 基于同义词词典的方法

要表示单词含义，首先可以考虑通过人工方式来定义单词含义，一种方式就是像词典一样，一个词一个词地说明单词含义。NLP 中广泛运用的一种词典是同义词词典 (thesaurus)，在词典中，含义类似的单词被归类到同一个组中。比如，car 的同义词有 automobile、motorcar 等。另外，在同义词词典中，有时还会定义更多的关系，比如上位下位关系、整体部分关系等。在 NLP 领域，最著名的同义词词典是普林斯顿大学于 1985 年开始开发的同义词词典 WordNet。

同义词词典有许多缺陷，第一点是它难以实时更新，随着时间的推移，会有新词不断出现，也会有旧词被赋予新的含义，同义词词典很难反映出这种变化。第二点是生成同义词词典的人力成本高，以英文为例，据说现有的英文单词总数超过 1000 万个，光靠人力创造完全的同义词词典不太现实。第三点是它无法表示单词的微妙差异，含义相近的单词之间可能也会有细微的区别，这些区别同义词词典不能很好体现。

5.3 基于统计的方法

基于统计的方法的目标是将单词表示成向量形式，在 NLP 中也称为分布式表示，这个想法基于分布式假设 (distributional hypothesis)：“某个单词的含义由它周围的单词形成”，也就是说单词含义可以它所在的上下文表示。该方法需要使用语料库 (corpus)，即大量的文本数据，如 Wikipedia 等。考虑句子：“I say yes and you say no.”，首先对文本进行分词，记录下每个单词的上下文（假设窗口大小为 1），I 的上下文只有 say，其表示为：

	I	say	yes	and	you	no	.
I	0	1	0	0	0	0	0

即 you 向量表示为 (0, 1, 0, 0, 0, 0, 0)，继续处理剩余单词。最后生成一个生成共现矩阵 (co-occurrence matrix)，矩阵每行对应一个单词向量：

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

在上面的矩阵中，元素表示两个单词同时出现的次数，这种表示有时可能出现问题。考虑语料库频繁出现 “...the car...” 的情况，如果只看单词的出现次数，有可能出现 the 和 car 的相关性比 car 与 drive 相关性更强的情况。为了解决这一问题，可以使用点互信息（Pointwise Mutual Information, PMI）指标，对于随机变量 x 和 y ，它们的 PMI 定义如下：

$$\text{PMI}(x, y) = \log \frac{P(x, y)}{P(x)P(y)}$$

为了避免 $\log 0$ ，实践上会使用正的点互信息（Positive PMI, PPMI）：

$$\text{PPMI}(x, y) = \max(0, \text{PMI}(x, y))$$

5.4 基于推理的方法

基于统计的方法在处理大规模语料库会出现问题，由于单词数量非常大，生成的矩阵也会非常庞大，处理起来不太方便。本节介绍基于推理的方法，他的原理相当于训练一个嵌入模型，让它做完形填空，通过反复的预测训练，学习到单词的嵌入表示。

在使用神经网络处理单词之前，首先需要将单词转换为向量表示。其中最简单的一种方式是使用 one-hot 编码。这种表示方法中，向量的长度等于词汇表的大小，且仅将对应单词 ID 的位置设为 1，其余位置均为 0。

Word2Vec 是 Google 于 2013 开源的词向量计算工具，其中使用的模型之一为 CBOW (continuous bag-of-words) 模型。CBOW 模型的输入是上下文，这个上下文用单词列表表示，每个单词用 one-hot 表示成向量，因此如果上下文考虑 N 个单词，则输入层会有 N 个。CBOW 中，中间层的神经元是各个输入层经全连接层变换后得到的值的平均，输出层神经元数目等于词表大小，这些神经元是各个单词的得分，它的值越大，说明对应单词的出现概率就越高。

Word2Vec 中还有一个是被称为 skip-gram 的模型，skip-gram 反转了 CBOW 模型的目标，CBOW 模型从上下文预测单词，而 skip-gram 模型则从单词预测上下文。在 Word2Vec 中，模型的输入侧的中间层权重就是单词的分布式表示。

考虑 one-hot 表示，由于其中只有一个为 1 的元素，它与中间层权重矩阵的乘积其实就是取矩阵的某一行，也即获得单词的分布式表示，这个分布式表示也被称为词嵌入 (word embedding)，所以输入侧中间层有时也被称为 Embedding 层。由于这个操作比较简单，实践上一般用查表的方式代替矩阵乘积。

6 分词

6.1 分词简介

要对文本进行处理，第一件事就是分词（Tokenization），将文本切分成一个个 Token，Token 可以是单词、子词或字符等。分词的方式有很多种，比较简单的方式是按字、按词切分，复杂一点的方式是 n-gram 分词，切分时 n 个 Token 为一组来对文本进行切分。

- 文本：人工智能让世界变得更美好
- 按字：人 工 智 能 让 世 界 变 得 更 美 好
- 按词：人 工 智 能 让 世 界 变 得 更 美 好
- 按字 Bi-Gram：人/工 工/智 智/能 能/让 让/世 世/界 界/变 变/得 得/更 更/美 美/好 好/
- 按词 Bi-Gram：人工智能/让 让/世界 世界/变得 变得/更 更/美好 美好/

由于分词是按词表进行的，词表的产生就是一个重要的工作，由此产生了多种分词算法，常见的有 BPE、BBPE、WordPiece 等。

6.2 BPE

先介绍 BPE（byte pair encoding）的工作原理，假设有一个语料库，我们首先从其中提取所有的单词并计算它们出现的次数。假设提取出来的单词和计数是：

(cost, 2)、(best, 2)、(menu, 1)、(men, 1)、(camel, 1)

先将所有的单词变成字符序列，用出现过的所有字符初始化词表，此时词表的大小为 11。

a, b, c, e, l, m, n, o, s, t, u

接下来，我们需要定义词表迭代的终止条件，假设我们要创建大小为 14 的词表。为了在词表中添加一个新 Token，我们需要先确定出现得最频繁的符号对，将它们合并之后添加到词表中，重复这一过程，直到达到词表的大小要求。

延续前面的例子，从字符序列可以看出，出现得最频繁的符号对是 s 和 t，符号对 s 和 t 出现了 4 次（两次是在 cost 中，另外两次是在 best 中），因此将 st 添加到词表。

a, b, c, e, l, m, n, o, s, t, u, st

下面，重复同样的步骤，再次检查出现的最频繁的符号对，可以计算出现在出现得最频繁的符号对是 m 和 e，这个符号对出现了 3 次（menu、men、camel）。将 m 和 e 合并，并将其添加到词表中，继续以上步骤，直到词表大小为 14，最终词表如下所示：

a, b, c, e, l, m, n, o, s, t, u, st, me, men

BPE 所涉及的步骤小结如下:

1. 从给定的语料集中提取单词并计算它们出现的次数。
2. 确定词表的大小。
3. 将单词拆分成一个字符序列。
4. 将字符序列中的所有非重复字符添加到词表中。
5. 选择并合并具有高频率的符号对。
6. 重复步骤 5, 直到达到步骤 2 中所设定的词表大小。

6.3 BBPE

BBPE (byte-level byte pair encoding, BBPE) 的工作原理与 BPE 非常相似, 但它不使用字符序列, 而是使用字节序列。假设输入文本只有单词 best, 在 BPE 中, 需要将单词转换为字符序列: b e s t, 而在 BBPE 中, 我们不是将单词转换为字符序列, 而是将其转换为字节序列: 62 65 73 74, 假设输入是“你好”这个中文词组, 则会被转换为字节序列: e4 bd a0 e5 a5 bd。BBPE 的好处是字节表示的通用性使其能处理任何编码的文本。

6.4 WordPiece

WordPiece 和 BPE 主要的不同之处在于合并对的选择方式。WordPiece 不是选择频率最高的字符对, 而是对每个字符对计算一个得分, 使用公式:

$$\text{score} = \frac{\text{freq_of_pair}}{\text{freq_of_first_element} \times \text{freq_of_second_element}}$$

7 RNN

7.1 语言模型

再考虑一下 word2vec 的 CBOW 模型，假设单词序列为 $w_1, w_2, \dots, w_T$ 且窗口大小为 1，它所做的事情就是从上下文 w_{t-1} 和 w_{t+1} 预测目标词 w_t 。当给定上下文时目标词是 w_t 的概率为 $P(w_t | w_{t-1}, w_{t+1})$ ，CBOW 模型就是对这一后验概率进行建模，如果使用交叉熵误差，其损失函数是 $L = -\log P(w_t | w_{t-2}, w_{t-1})$ 。在上一章我们使用 CBOW 模型的权重获得单词的表示，那它本来的目的“从上下文预测目标词”是否能用来做点事情呢？这就得提到语言模型了。

语言模型 (language model) 使用概率来评估一个单词序列发生的可能性。比如，对于 “you say goodbye” 这一单词序列，语言模型给出高概率；对于 “you say good die” 这一单词序列，模型则给出低概率。典型的语言模型应用有机器翻译和语音识别，比如，语音识别系统会根据人的发言生成多个句子作为候选，此时，使用语言模型，可以按照单词序列发生的可能性对候选句子进行排序。用数学表示语言模型：

$$\begin{aligned} P(w_1, \dots, w_m) &= P(w_m | w_1, \dots, w_{m-1}) \cdot P(w_{m-1} | w_1, \dots, w_{m-2}) \cdots P(w_2 | w_1) \cdot P(w_1) \\ &= \prod_{t=1}^m P(w_t | w_1, \dots, w_{t-1}) \end{aligned}$$

假设我们将 CBOW 模型改造成语言模型，那么首先得将原来对称的上下文改成只有目标词左侧的上下文，这样可以近似实现语言模型：

$$P(w_1, \dots, w_m) = \prod_{t=1}^m P(w_t | w_1, \dots, w_{t-1}) \approx \prod_{t=1}^m P(w_t | w_{t-2}, w_{t-1})$$

不过这个实现还是有点问题，首先是它只能处理固定长度的上下文，会有信息丢失；其次是上下文的单词顺序会被忽视，比如，(you, say) 和 (say, you) 会被作为相同的内容进行处理。

7.2 RNN 介绍

循环神经网络 (Recurrent Neural Network, RNN) 就可以很好地解决上述问题。之前我们看到的神经网络都是前馈型神经网络，网络的传播方向是单向的，虽然前馈网络结构简单、易于理解，但它不能很好地处理时间序列数据。

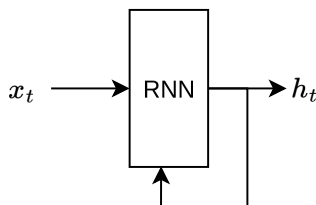


图 7.1: RNN 层示例

RNN 层有环路，通过该环路，数据可以在层内循环。时序数据 $(x_0, x_1, \dots, x_t, \dots)$ 被输入到层中。然后，以与输入对应的形式，输出 $(h_0, h_1, \dots, h_t, \dots)$ 。RNN 有两个权重，分别是将输入 x 转化为输出 h 的权重 W_x 和将前一个 RNN 层的输出转化为当前时刻的输出的权重 W_h 。RNN 的计算用数学表示为： $h_t = \tanh(h_{t-1}W_h + x_tW_x + b)$ 。RNN 的输出 h_t 称为隐藏状态 (hidden state) 或隐藏状态向量 (hidden state vector)。

由于 RNN 层有环路，因此不能直接使用前馈神经网络的反向传播算法，RNN 的方向传播是基于时间的反向传播 (Back Propagation Through Time, BPTT)，具体做法是展开 RNN 的循环，把它看作普通的神经网络，然后运用之前的反向传播方法即可。

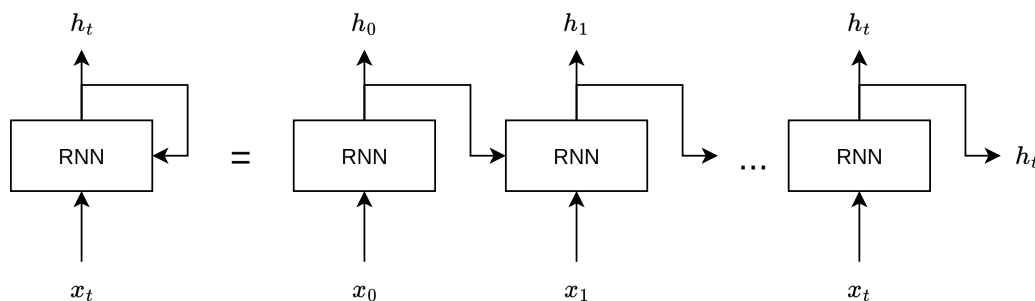


图 7.2: RNN 的展开

在处理长度为 1000 的时序数据时，如果展开 RNN 层，它将成为在水平方向上排列有 1000 个层的网络，序列太长的话，会出现计算量或者内存使用量方面的问题，此外，随着层变长，梯度逐渐变小，梯度将无法向前一层传递。因此，考虑将时间轴方向上过长的网络在合适的位置进行截断，从而创建多个小型网络，然后对截出来的小型网络执行误差反向传播法，这个方法称为 Truncated BPTT。

在 RNN 的学习中，为了执行 mini-batch 学习，需要将输入数据进行“偏移”。对长度为 1000 的时序数据，如果将批大小设为 2，那么第 1 笔样本数据从头开始按顺序输入，第 2 笔数据就从第 500 个数据开始按顺序输入。

7.3 语言模型的评估

困惑度 (perplexity) 常被用作评价语言模型的预测性能的指标，困惑度表示“概率的倒数”。考虑 “you say goodbye and i say hello.” 语料库，假设在向语言模型传入单词 you 时会输出的概率分布中，单词是 say 的概率为 0.2，那么困惑度就是 $1/0.2=5$ 。困惑度可以解释为下一个可以选择的选项的数量，在刚才的例子中，模型的困惑度是 5，这意味着下一个要出现的单词的候选个数为 5 个。在输出数据为多个的情况下，困惑度计算公式为： $Perplexity = e^L$ ，其中，L 为交叉熵损失函数的值。

7.4 梯度消失和梯度爆炸

前面介绍的简单的 RNN 在无法很好地学习到时序数据的长期依赖关系。RNN 中使用了函数 $\tanh(x)$ ，它的导数为 $1 - \tanh^2(x)$ ，这个值小于 1，并且随着 x 远离 0，它的值在变小，这就导致 RNN 在处理长序列时容易出现梯度消失的问题。另外，RNN 中使用了矩阵乘积，可能导致梯度消失或者梯度爆炸，这里不具体分析了。

梯度爆炸的问题可以通过梯度裁剪 (gradients clipping) 来解决，

$$g \leftarrow \begin{cases} g, & \text{if } \|g\|_2 \leq \tau \\ g \cdot \frac{\tau}{\|g\|_2}, & \text{if } \|g\|_2 > \tau \end{cases}$$

这里假设可以将神经网络用到的所有参数的梯度整合成一个向量 g ，且阈值设置为 τ 。

7.5 LSTM

在 RNN 的学习中，梯度消失也是一个大问题，为了解决这个问题，LSTM (Long Short-Term Memory) 中增加了一种名为“门”的结构，并且 LSTM 中有专门的权重参数用于控制门的开合程度，这些权重参数也会在学习中被更新。在 LSTM 中存在记忆单元 c ，仅在 LSTM 层内部接收和传递数据，不向其他层输出。其中 $h_t = \tanh(c_t)$ 。

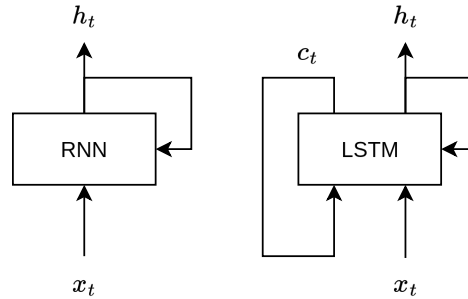


图 7.3: RNN 与 LSTM

输出门 (output gate) 针对 $\tanh(c_t)$ 的元素，调整它们作为下一时刻的隐藏状态的重要程度。输出门的开合程度根据输入 x_t 和上一个状态 h_{t-1} 求出 (σ 表示 sigmoid 函数)，最后将这个 o_t 和 $\tanh(c_t)$ 的对应元素的乘积作为 h_t 输出。:

$$o_t = \sigma(x_t W_x^{(o)} + h_{t-1} W_h^{(o)} + b^{(o)})$$

遗忘门 (forget gate) 是一个忘记不必要记忆的门，遗忘门的输出 f_t 与记忆单元 c_{t-1} 相乘，生成新的记忆单元 c_t :

$$f_t = \sigma(x_t W_x^{(f)} + h_{t-1} W_h^{(f)} + b^{(f)})$$

遗忘门用来遗忘记忆，现在我们还想向记忆单元添加一些应当记住的新信息，为此我们添加新的 $\tanh$ 节点，通过将这个 g_t 加到上一时刻的 c_{t-1} 上，从而形成新的记忆：

$$g_t = \tanh(x_t W_x^{(g)} + h_{t-1} W_h^{(g)} + b^{(g)})$$

最后添加一个输入门 (input gate)，将 i_t 和 g_t 的对应元素的乘积添加到记忆单元中：

$$i_t = \sigma(x_t W_x^{(i)} + h_{t-1} W_h^{(i)} + b^{(i)})$$

因为在 LSTM 中，记忆单元的更新仅有 $+$ 和 $\times$ 运算，因此梯度消失和梯度爆炸问题都得到有效解决。

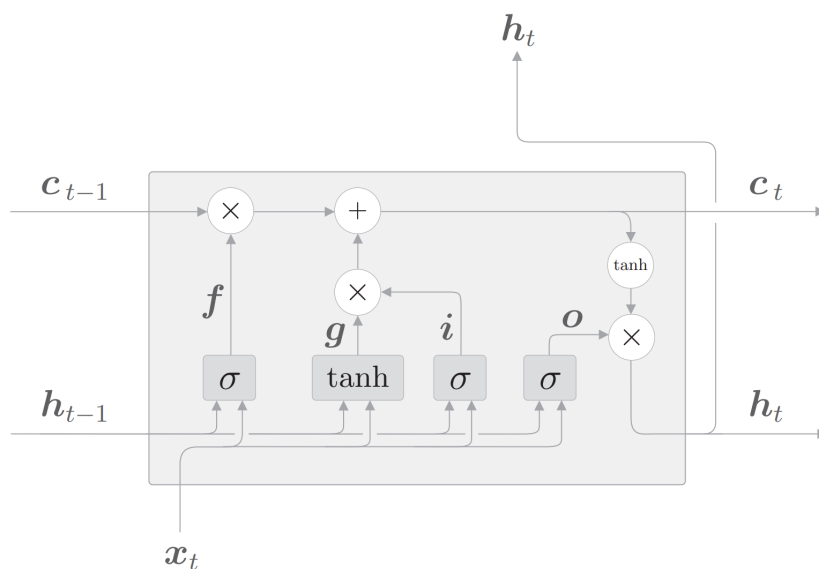


图 7.4: LSTM 的计算过程

8 文本生成

8.1 Seq2Seq 模型

语言模型根据已经出现的单词输出下一个出现的单词的概率分布，通过对这个概率分布进行采样，就可以逐步生成完整的文本。最常见的采样方法是每次选择概率最高的词作为下一个输出，重复该过程，直到生成结束符号 $\langle \text{eos} \rangle$ 为止。

Seq2Seq 模型则是一类用于将一种序列转换为另一种序列的模型，广泛应用于机器翻译、语音识别等任务。它通常由两个模块组成：Encoder 和 Decoder，因此也被称为 Encoder-Decoder 模型。编码器将输入序列编码为一个向量表示，常见结构是 Embedding 层加 RNN 层；解码器则根据这个向量表示逐步生成输出序列，其生成过程可以采用前面提到的采样策略。

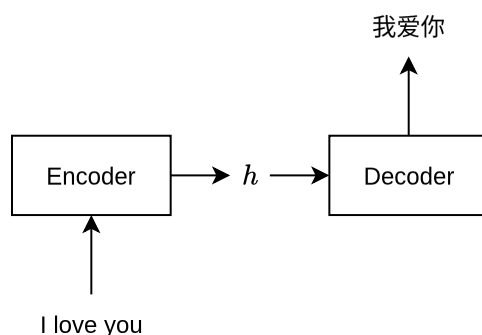


图 8.1: Seq2Seq 模型示例

编码器的输出是固定长度的向量，但输入序列的长度是可变的，这可能会导致部分信息丢失。为了解决这个问题，我们可以使用编码器在每个时间步的隐藏状态，将其拼接成一个矩阵，作为解码器的输入。由于解码器原本只接收一个向量输入，因此也需要相应地进行调整。我们可以通过对该矩阵的行向量进行加权平均来将其转换为向量，而加权平均中权重的确定，就是注意力机制 (Attention Mechanism) 所解决的问题。

8.2 Self-Attention

在基于 RNN 的 Seq2Seq 模型中，注意力机制使得解码器在生成每个输出单词时，能够动态地关注输入序列中的不同部分。例如，在将 “I love you” 翻译为 “我爱你” 的过程中，生成 “我” 时模型更倾向于关注 “I”，而生成 “爱” 时则更多关注 “love”。既然输出序列可以使用注意力机制，那么输入序列是否也可以使用注意力机制呢？

答案是肯定的，这正是 Transformer 模型中自注意力机制 (Self-Attention) 的作用。需要说明的是，Transformer 作为 Seq2Seq 模型的一种实现方式，也包含类似于基于 RNN 的 Seq2Seq 模型中的注意力机制，这部分在 Transformer 中被称为交叉注意力 (Cross-Attention)。让我们通过一个例子来快速理解自注意力机制，例句：

A dog ate the food because it was hungry

显然，例句中的代词 it 指代的是 dog 而不是 food，自注意力机制的作用就是关注句子间单词的关系。

将句子输入模型的编码器，得到一个输出矩阵 $\mathbf{X}$ ，为了计算自注意力，我们需要再创建三个新的矩阵：查询（query）矩阵 $\mathbf{Q}$ 、键（key）矩阵 $\mathbf{K}$ ，以及值（value）矩阵 $\mathbf{V}$ 。为了创建它们，我们需要先创建另外三个权重（weight）矩阵， $\mathbf{W}^Q$ 、 $\mathbf{W}^K$ 、 $\mathbf{W}^V$ 。用矩阵 $\mathbf{X}$ 分别乘以矩阵 $\mathbf{W}^Q$ 、 $\mathbf{W}^K$ 、 $\mathbf{W}^V$ ，依次创建出 $\mathbf{Q}$ 、 $\mathbf{K}$ 和 $\mathbf{V}$ 。值得注意的是，权重矩阵 $\mathbf{W}^Q$ 、 $\mathbf{W}^K$ 和 $\mathbf{W}^V$ 的初始值完全是随机的，但最优值则需要通过训练获得。我们取得的权值越优，通过计算所得的查询矩阵、键矩阵和值矩阵也会越精确。

假设输入句子为 “I am good”，自注意力机制首先要计算查询矩阵 $\mathbf{Q}$ 与键矩阵 $\mathbf{K}^T$ 的点积。

$$\mathbf{Q} \cdot \mathbf{K}^T = \begin{bmatrix} \mathbf{q}_1 \cdot \mathbf{k}_1 & \mathbf{q}_1 \cdot \mathbf{k}_2 & \mathbf{q}_1 \cdot \mathbf{k}_3 \\ \mathbf{q}_2 \cdot \mathbf{k}_1 & \mathbf{q}_2 \cdot \mathbf{k}_2 & \mathbf{q}_2 \cdot \mathbf{k}_3 \\ \mathbf{q}_3 \cdot \mathbf{k}_1 & \mathbf{q}_3 \cdot \mathbf{k}_2 & \mathbf{q}_3 \cdot \mathbf{k}_3 \end{bmatrix}$$

首先，来看 $\mathbf{Q} \cdot \mathbf{K}^T$ 矩阵的第一行，可以看到，这一行计算的是查询向量 $\mathbf{q}_1$ (I) 与所有的键向量 $\mathbf{k}_1$ (I)、 $\mathbf{k}_2$ (am) 和 $\mathbf{k}_3$ (good) 的点积。通过计算两个向量的点积可以知道它们之间的相似度。因此，通过计算查询向量 ($\mathbf{q}_1$) 和键向量 ($\mathbf{k}_1$ 、 $\mathbf{k}_2$ 、 $\mathbf{k}_3$) 的点积，可以了解单词 I 与句子中的所有单词的相似度。我们了解到，I 这个词与自己的关系比与 am 和 good 这两个词的关系更紧密，因为点积值 110 大于 90 和 80。

$$\mathbf{Q} \cdot \mathbf{K}^T = \begin{bmatrix} 110 & 90 & 80 \\ 70 & 99 & 70 \\ 90 & 70 & 100 \end{bmatrix}$$

表 8.1: 点积示例，本节的结果均为假设值，后续不再说明

自注意力机制的第 2 步是将 $\mathbf{Q} \cdot \mathbf{K}^T$ 矩阵除以键向量维度的平方根，这样做的目的主要是获得稳定的梯度。

目前所得的相似度分数尚未被归一化，所以第 3 步是使用 softmax 函数对其进行归一化处理。应用 softmax 函数将使数值分布在 0 到 1 的范围内，且每一行的所有数之和等于 1。这个矩阵被称为分数 (score) 矩阵，通过它，我们可以了解句子中的每个词与所有词的相关程度。比如：I 这个词与它本身的相关程度是 90%，与 am 这个词的相关程度是 7%，与 good 这个词的相关程度是 3%。

$$\text{softmax}\left(\frac{\mathbf{Q} \cdot \mathbf{K}^T}{\sqrt{d_k}}\right) = \begin{bmatrix} & \text{I} & \text{am} & \text{good} \\ \text{I} & 0.90 & 0.07 & 0.03 \\ \text{am} & 0.025 & 0.95 & 0.025 \\ \text{good} & 0.21 & 0.03 & 0.76 \end{bmatrix}$$

自注意力机制的最后一步是计算注意力矩阵 $\mathbf{Z}$ ，注意力矩阵包含句子中每个单词的注意力值。它通过将分数矩阵乘以值矩阵 $\mathbf{V}$ 得出。

$$\begin{aligned}
Z &= \text{softmax} \left(\frac{Q \cdot K^T}{\sqrt{d_k}} \right) V \\
&= \begin{bmatrix} 0.90 & 0.07 & 0.03 \\ 0.025 & 0.95 & 0.025 \\ 0.21 & 0.03 & 0.76 \end{bmatrix} \begin{bmatrix} 67.85 & 91.2 & \dots & 0.13 \\ 13.13 & 63.1 & \dots & 4.44 \\ 12.12 & 96.1 & \dots & 43.4 \end{bmatrix} \\
&= \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix}
\end{aligned}$$

可以看出，单词 I 的自注意力值 z_1 是分数加权的值向量之和。所以， z_1 的值将包含 90% 的值向量 v_1 (I)、7% 的值向量 v_2 (am)，以及 3% 的值向量 v_3 (good)。

$$z_1 = 0.90 \begin{bmatrix} 67.85 & 91.2 & \dots \end{bmatrix} + 0.07 \begin{bmatrix} 13.13 & 63.1 & \dots \end{bmatrix} + 0.03 \begin{bmatrix} 12.12 & 96.1 & \dots \end{bmatrix}$$

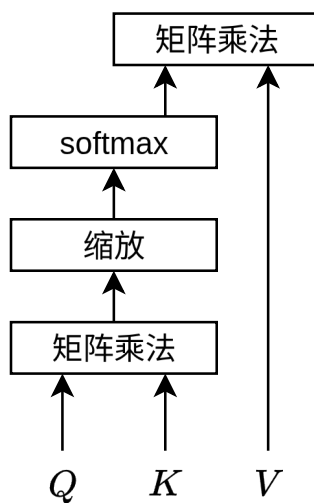


图 8.2: 自注意力计算流程

9 Transformer

9.1 位置编码

在 RNN 中，句子是逐词送入网络的，以 I am good 为例，首先把 I 作为输入，接下来是 am，最后是 good。然而，Transformer 并不遵循这个模式，它将句子中的所有词并行地输入到神经网络中。由于是并行输入，所以词序需要通过其他方式表示，Transformer 所采用的方式是位置编码。

在 Transformer 中，位置编码以矩阵的形式表示，位置编码矩阵 P 的维度与输入矩阵 X 的维度相同。在将输入矩阵直接传给 Transformer 之前，我们需将位置编码矩阵 P 加到输入矩阵 X 中，再将其作为输入送入网络。Transformer 中使用了正弦函数来计算位置编码：

$$P(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \quad P(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

在上面的等式中，pos 表示该词在句子中的位置， i 表示在输入矩阵中的位置。

$$P = \begin{bmatrix} \sin\left(\frac{\text{pos}}{10000^0}\right) & \cos\left(\frac{\text{pos}}{10000^0}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \\ \sin\left(\frac{\text{pos}}{10000^1}\right) & \cos\left(\frac{\text{pos}}{10000^1}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \\ \sin\left(\frac{\text{pos}}{10000^0}\right) & \cos\left(\frac{\text{pos}}{10000^0}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \end{bmatrix}$$

9.2 编码器与解码器

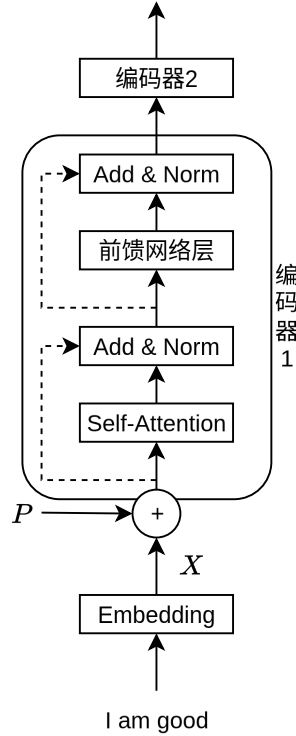


图 9.1: Transformer 编码器结构

Transformer 的编码器由一组 N 个编码器串联而成，每一个编码器的构造都是相同的，由自注意力层和前馈网络层组成，每层的输出都经过残差连接和归一化层（Add & Norm）。前馈网络由多个全连接层组成，一般采用 ReLU 激活函数。

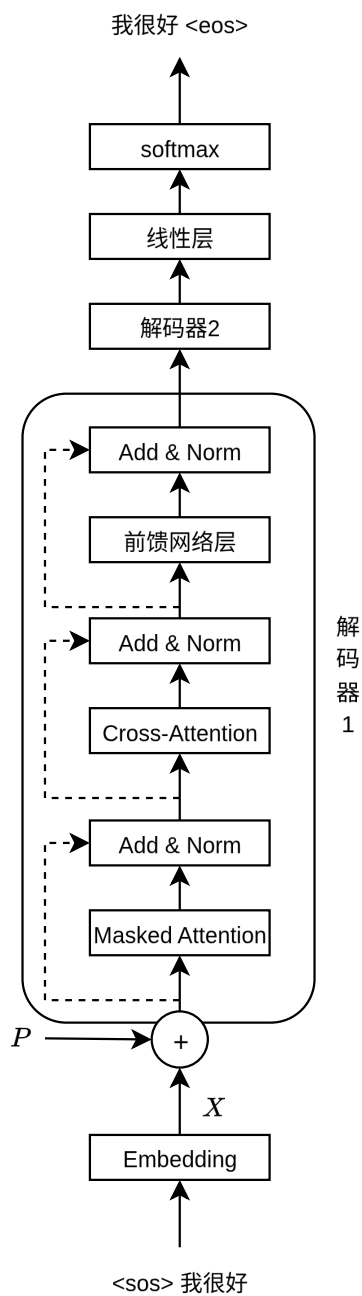


图 9.2: Transformer 解码器结构

相较编码器，解码器的注意力层不太一样，它需要关注编码器的输出，因此解码器的注意力层

分为两部分：带掩码的注意力层和交叉注意力层。交叉注意力层都有两个输入：一个来自带掩码的注意力层，另一个是编码器的输出。让我们用 $\mathbf{R}$ 来表示编码器的输出值，用 $\mathbf{M}$ 来表示由带掩码的注意力层输出的注意力矩阵，注意力的计算使用矩阵 $\mathbf{M}$ 创建查询矩阵 $\mathbf{Q}$ ，使用 $\mathbf{R}$ 创建键矩阵和值矩阵。在解码器中的带掩码的注意层中，我们对注意力计算中的矩阵进行掩码处理，确保解码器在生成每个单词时只能关注之前的单词。

$$\frac{\mathbf{Q}_i \cdot \mathbf{K}_i^T}{\sqrt{d_k}} = \begin{bmatrix} & \text{<sos>} & \text{我} & \text{很} & \text{好} \\ \text{<sos>} & 9.125 & -\infty & -\infty & -\infty \\ \text{我} & 5.0 & 12.37 & -\infty & -\infty \\ \text{很} & 7.25 & 5.0 & 10.37 & -\infty \\ \text{好} & 1.5 & 1.37 & 1.87 & 10.0 \end{bmatrix}$$

获得解码器的输出之后，我们将其送入线性层和 softmax 层，线性层将生成一个 logit 向量，其大小等于原句中的词汇量，然后，将 softmax 函数应用于 logit 向量，从而得到概率分布，最后对其进行采样得到最终的文本输出。

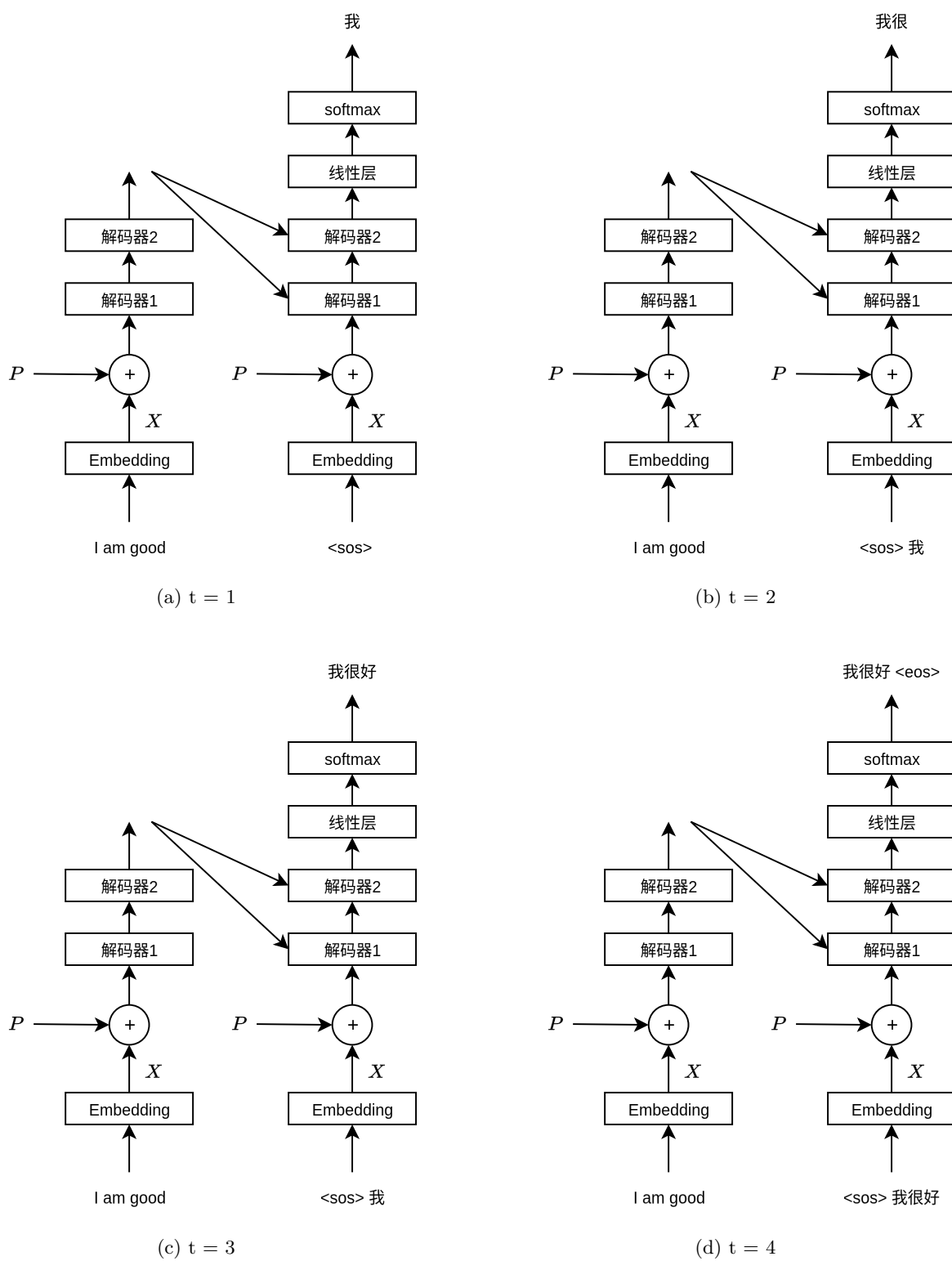


图 9.3: 解码器流程说明

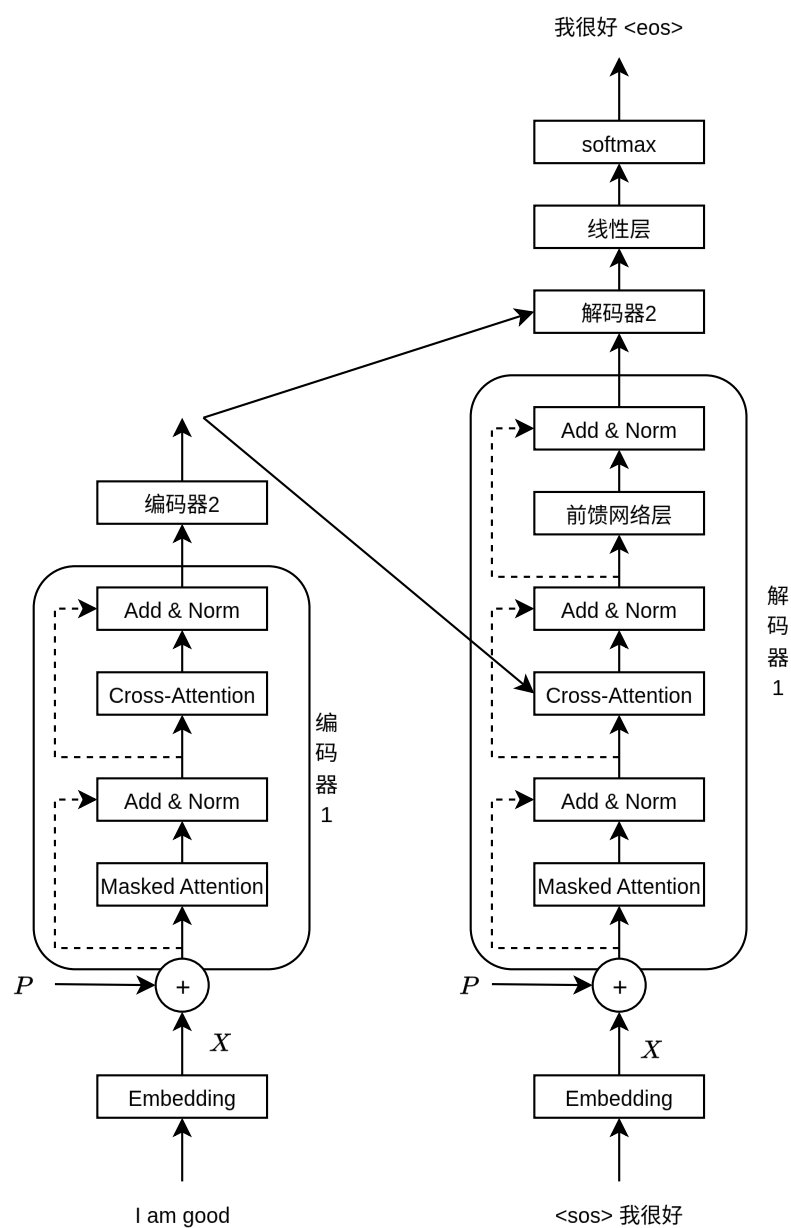


图 9.4: Transformer 总结构